MŰEGYETEM 1782

Témalabor beszámoló

Távközlési és Médiainformatikai Tanszék

Készítette:	**Bacsó Janka**
Neptun-kód:	**A6VVDJ**
Ágazat:	**Infokommunikáció**
E-mail cím:	**janka.bacso@edu.bme.hu**
Konzulens(ek):	**Papp Dávid**
E-mail címe(ik):	**pappd@tmit.bme.hu**

Téma címe: AlexNet Architektúra ismertetése és alkalmazása a CIFAR-10 adathalmazon

Feladat

A kiadott feladat úgy szólt, hogy egy általunk választott CNN architektúrának az alkalmazását a CIFAR-10 adathalmazon kell bemutatni/ismertetni. Az én választásom a AlexNet-re esett. A munkát bemutató részletes .ipynb készítése ajánlott, a dokumentáció mellé. A beadandó tartalmazza a hiba nélkül futtatható kódblokkokat, a kódblokkok működésének magyarázatát (ahol szükséges).

2022/2023. 1. félév

1. A laboratóriumi munka környezetének ismertetése, a munka előzményei és kiindulási állapota

1.1 Bevezető

A félév során megismerkedtünk a témalabor keretein belül a képfeldolgozás folyamatával és különféle osztályozókkal. A laborok végére az összes szükséges lépésen végig mentünk, mint például leírok kinyerése, klaszterezés, osztályozás. A feladat, amit kaptunk ennek a tudásnak való felhasználása egy állatunk választott CNN architektúrával értelmezése és bemutatása során.

1.2 Elméleti összefoglaló

Az AlexNet 2012-ben lett puklikával Alex Krizhevsky, Ilya Sutskever és Geoffery Hinton által. Ez az ImageNet nevű dataset osztályozására lett kitalálva. Akkoriban egy olyan probléma állt fenn, hogy minél nagyobb adathalmazt szerettek volna osztályozni annál több réteget kellett volna létrehozni, ami sok erőforrást igényelt. Az Alexnet-nek szimplán 8 réteg segítségével sikerült osztályoznia az ImageNet-et.

Az egyik fő szempont, amit maguk elött tartottak az AlexNet kitalálói, hogy a túl tanulást a lehető legjobban meg kéne akadályozni. A „túl tanulás” egy olyan jelenség, mikor a gépi tanulás során az algoritmus eléri azt a pontot, hogy bár nagyon pontos értéket ad, de csak a tanulás során alkalmazott adatokra. Más szóval betanulta őket.

Három módszert említ meg a dokumentáció, amit túl tanulás ellen használtak: Maxpooling, Dropout, Data Augmentation.

Maxpooling-nak egy overlapping verzióját használjak. A pooling metódus alapjáraton arra szolgál, hogy csökkentsük az adat méretét. A módszer során a kimeneti adat egy rácspontja, egy nagyobb területet szumáz össze a bemenetből. Overlapping esetében az általánosítás során a léptető ablak lépései, amivel végig haladunk a képen, átfedésbe leszeknek.

Dropout egy olyan metódust hív meg, aminek során minden neuron kap egy valószínűségi szorzót, hogy kiesik-e az architektúrából a képfeldolgozás során vagy nem. Kiesés azt jelenti, nem vesz részt az adat osztályozás/elemzés során. Ez minden adatnál változik, így a különböző adatok, az architektúra különböző verziói alapján lesznek osztályozva. Így sokkal nehezebb egy bizonyos értékre beállnia a rendszernek. A különböző verziók súlyai viszont ugyanzok maradnak.

Data Augmentation az a folyamat, mikor az adatot/képet megváltoztatva szintén beadjuk újra a gépnek. A változás lehet forgatás, méret változás, egy rész kivágása és annak vizsgálata. Ez azért kell, hogy például a kutyát ne csak akkor ismerje fel ha egy helyben ül a kép közepén.

Fent említettem hogy az AlexNet 8 rétegből áll : 5db convolutional rétegből és 3db fully connected rétegből. Emellett 3 maxpooling is van még az architektúrában. (*kif. 2. részben*)

1.3 A munka állapota, készültségi foka a félév elején

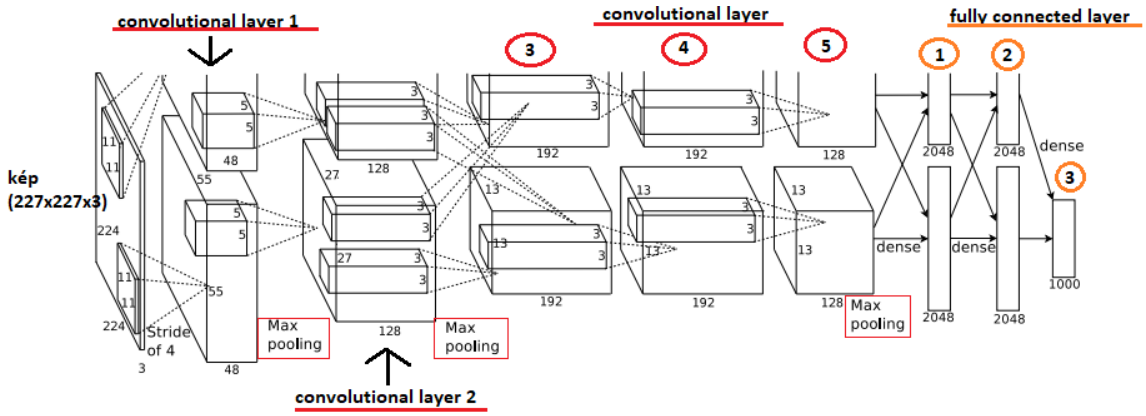
Félév elején még nem sok fogalmam volt a témáról. Előző évben elvégeztem a *Mesterséges Intelligencia* nevű kötelező tárgyat, aminek keretében nagyon alap működését megtanultam a deep learning-nek. Minden mással a témalabor keretén belül találkoztam.

2. Az elvégzett munka és eredmények ismertetése

A feladat során kerestem egy kódot, ami az AlexNet architektúrát implementálj. Ha szükséges volt kijavítottam, majd saját környezetben teszteltem és értelmeztem. A továbbiakban a kód működésének magyarázat lesz látható.

2.1 Modell megépítése

A kód, amit én használok valamennyiben eltér az eredetitől, ugyanis az eredeti az ImageNet dataset-re lett kitalálva. Nekem a CIFAR-10-re kellett megoldanom a feladatot, ami a képek méretében és az osztályok számában eltér.



A keras *Sequential* modell példányosításával kezdődik a kód. Ez egy olyan modell, amibe egymás után beletesszük, majd a rétegjeinket.

2.1.1 Konvoluciós rétegek

A kódban az első réteg egy konvoluciós réteg, ami adott méretű és beállítású filterek segítségével információt szed ki egy képből. A filtereknek különböző beállításaik lehetnek. Ezzel különböző attribútumokat próbálnak vizsgálni, hogy megtalálhatóak-e a képen. Például egy filter kereshet sarkokat, egy másik éleket szeretne találni. Állatok esetében, azok ismertető jeleit szedi ki különböző filterekkel. Egy konvolúciós réteg több filtert is használhat.
A mi esetünkben a következő kép néz ki a kód:

AlexNet.add(Conv2D(filters=96, input_shape=(32,32,3), kernel_size=(11,11), strides=(4,4), padding='same'))

Ez annyit tesz, hogy a 32x32x3(eredetiben 227x227x3) lesz a bemenő képünk mérete. A x3 az az RGB rétegeket jelöli. 96 filteret fog végig próbálni rajta. egy filter 11x11x3-as méretű ezeket 4x4-es **stride**-al(csúsztatással) lépteti.

A **stride** annyit jelent, hogy miután ráhelyeztük a képre a filterünket 4 rácspontot lép oldalra míg a szélhez nem ér. Majd 4-gyel rácsponttal lejjebb megy a képen és újra elkezd 4esével lépkedni. Ezt addig folytatja míg végig nem ér a képen. Ez 1db filter.

Padding= 'same' annyit takar, hogy a kép kap egy **0** értékekből álló keretet minden irányba. ez azért szükséges hogy a kép széleit is rendesen nem tudjuk vizsgálni.

Ezek után hívunk egy *BatchNormalization* -t, ami arra szolgál, hogy minden értékünk -1 és 1 közé kerüljön. Miért van erre szükségünk? Mert a ***Relu*** az aktivációs függvényünk.

Ez a függvény minden **0** alatti értéket **0**-ra állít ez által elhagyhatóvá teszi őket. Az aktivációs függvénynek az a feladata, hogy „eldöntse" egy neuron értéke fontos-e számunkra a folyamat során.

Ezután hívjuk meg az első maxpoolingot. A maxpooling nem csak a túltanulásban segít, hanem az adat méretének csökkentésében is.

Ezek ismeretében és az architektúra leírása alapján megépítjük a többi konvoluciós réteget. Maxpooling csak a második és az ötödk réteg után lesz meghíva.

2.1.2 Fully Connected rétegek

Mielőtt a fully connected layer[1]-eket megírhatnánk, hozzá adunk egy *Flatten* réteget. Ezzel a több dimeziós bemenetet, egy 1 dimenziós vektorrá alakítjuk. Ezután jön az első fully connected layer.

A *Dense* réteg az réteg ahol az összes neuron össze kapcsolódik(fully connceted). A benne megadott argumentum a kimeneti réteg méretét mutatja.

Utána pedig a konvoluciós rétegeknél megismert módon a **ReLu**-val kiemeljük a fontos értékeket. Végezetül meghívjuk a Dropout függvényt, ami 0.4-es valószínűséget ad minden neuronnak a kiesésre az architektúrából.

Az alap leírásban 3 réteg van. Ebben a megvalósításban van egy negyedik, mert a CIFAR-10 dataset-et használjuk. Az utolsó réteg le szűkíti a CIFAR 10 osztályára az adatunkat. A **softmax** függvényt használja, ami annyit tesz, hogy a 10 érték, amit a végén megkapunk azt reprezentálja mennyi az esélye az egyes osztályoknak (mennyi az esélye, hogy abba az osztályba tartozik). Ha összeadjuk az értékeket 1-et kell kapunk.

2.1.3 Compile

Ezután hívunk egy *compile*-t, ami összeállítja a modellt. A következő argumentumokat kapja meg: *loss, optimizer metrics.*

Loss: megkapja milyen loss függvény alkalmazzon, hogy meg kapja a gradients, amivel a súlyokat változtatni kell. Ebben az esetben ***keras.losses.categorical_crossentropy***. Ez a függvény cross-entropy loss-t számol a valós(true) és a *megjósolt/megtippelt*(predicted) között.

Optimizer: optimalizálja a bementei súlyokat úgy, hogy összehasonlítja a loss függvényt a predikcióval. Változtat a tanulási rátán(learning rate). A kódban az ***'adam'*** optimalizálót adtam meg.

Metrics: a modellünk teljesítmény kiértékeléséhez kell. Nekünk az ***'accuracy'*** értéket kapta. Ez hasonlítja össze a címmel(label) a „*megjósolt*" (prediction) eredményt.

2.2 Training

Miután elkészült a modell jön a betanítás folyamata:

1. Először is letöltjük a képeinket a CIFAR-10-ből és betöltjük őket tárolókba. x_*train* a tanító halmaz képei az y_*train* pedig annak címei(label).
2. Ezután szétválasztjuk a képeket egy validációs halmazra és egy tanuló halmazra.
3. *to_categorical* fügvény segítségével beállítjuk hogy a label ne 1 darab adat legyen, hanem szét legyen osztva a 10 külön álló címre(label).
4. Data Augmentation kezelése: *ImageDataGenerator* segítségével létrehozunk „generátorokat", amiket meghívunk az adathalmazainkra. Ez random megváltoztatja a képeket a továbbiakban.
5. **lrr** értéket, azért hozzuk létre, hogy figyelje a *val_accuracy* értéket, hogy mennyire jó a becslése a gépnek, tanítás közben. Miután a montírozásra beadott érték nem változik/javul egy adott számú epoch után, csökkenti a learning rate-et. (callback értékben adjuk meg ezt)
6. A paraméterek meg adásakor hívjuk meg a data augmetation generator-t is.
7. *fit* függvénybe beadunk minden paramétert. Mi a tanuló adatat, mi a validációs adat, hány epoch kell… Ez a tanító folyamat.
8. Sikeres futattás után pedig kirajzolhatjuk az eredményeinket.

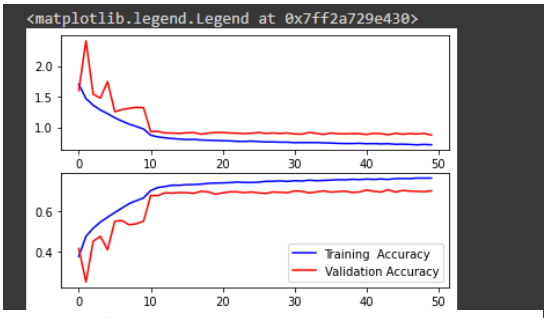
[1] Teljesen kapcsolt réteg a magyar megfelelője(nekem nem tetszik).

2.3 Eredmények

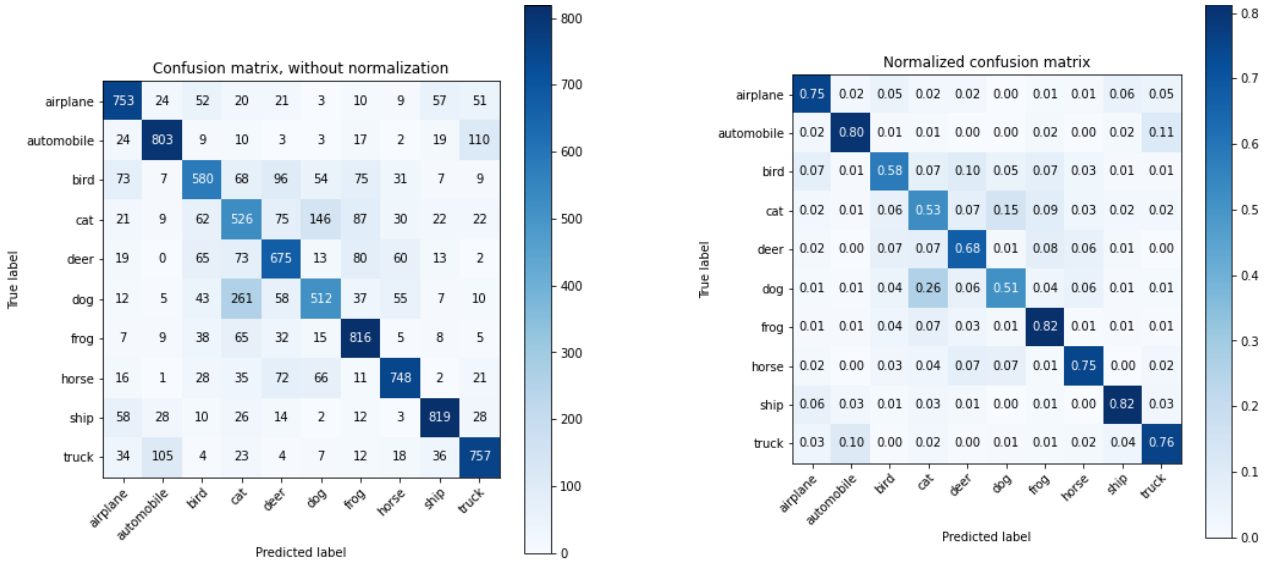
Az első ábra, amit látunk azt mutatja meg, hogy a validációs adathalmaz hogyan teljesített a training-hez képest. A képen egy igazán jó eredmény látható. Azt jelenti, hogy a modell egész pontos eredménnyel ismer fel képeket.

A második ábra egy konfúziós mátrix látható. A fő átló azt mutatja meg, hogy egy bizonyos kategóriában hány képet azonosítottunk helyesen. A legnagyobb tévesztést a kutyánál és macskánál lehet látni. Emellett az automobile és truck-nál is lehet látni, hogy 100 főlé megy tévesztés.

A modellünk végső Accuracy Score = 0.6989.



1. ábra

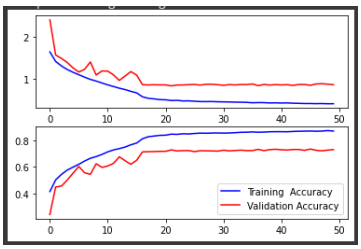


2. ábra

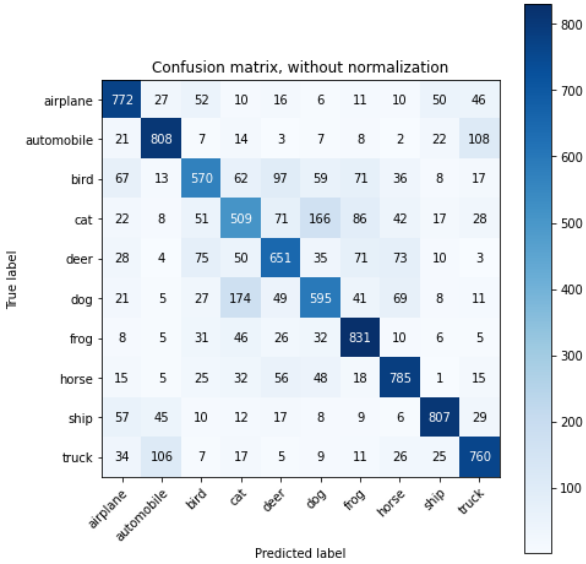
2.4 Optimalizálás változtatása

Nadam

A fent leírt modellben megváltoztattam az optimalizálás típusát. A következő értékeket kaptam rá:



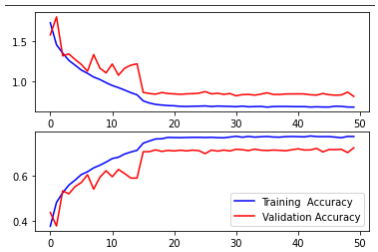

```
Accuracy Score =  0.7336
```



Ezen az látható, hogy a Nadam használatával jobb eredményt kapunk.

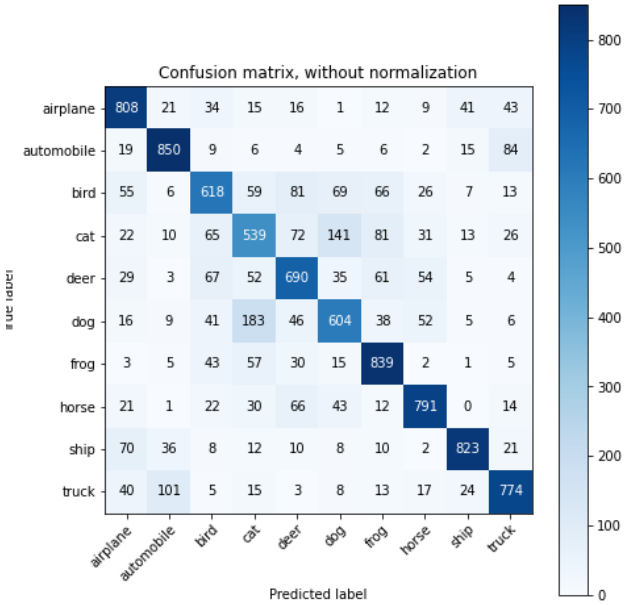
SGD

Egy harmadik optimalizálót is kipróbáltam:




```
Accuracy Score =  0.7088
```

Ez pedig mondhatni kettő közötti értékeket produkált. Enyhén jobb az adam-nál, de rosszam a Nadam-nál.



Ezekből az vonható le, hogy egy architektúra építése során gondosan kell megválasztani a paramétereinket a lehető legjobb eredmény érdekében.

2.5 Összefoglalás

A feladat által gyakorlatban is tanulmányoztam egy CNN architektúrát. Segített jobban megérteni és megismerni a képfeldolgozás menetét. Szerintem elég felhasználó barát és könnyen értelmezhető, de emellett effektív architektúra. Bár most már léteznek jobbak, de hasznos lehet vele megismerkedni, ha valaki ebben a témában érdeklődik.

3. Irodalom, és csatlakozó dokumentumok jegyzéke

A tanulmányozott irodalom jegyzéke:

[1]	Alex Krizhevsky, Ilya Sutskever, Geoffery Hinton, *ImageNet Classification with Deep Convolutional Neural Networks, 2012,* https://papers.nips.cc/paper/2012/file/c399862d3b9d6b76c8436e924a68c45b-Paper.pdf
[2]	Kumar, Dr. Vaibhav. "Hands-on Guide to Implementing AlexNet With Keras for Multi-Class Image Classification." *Analytics India Magazine*, 11 June 2020, https://analyticsindiamag.com/hands-on-guide-to-implementing-alexnet-with-keras-for-multi-class-image-classification/
[3]	Alake, Richmond. "What AlexNet Brought to the World of Deep Learning." Medium, 10 July 2020, https://towardsdatascience.com/what-alexnet-brought-to-the-world-of-deep-learning-46c7974b46fc
[4]	Pujara, Abhijeet. "Concept of AlexNet:- Convolutional Neural Network." Medium, 9 June 2021, https://medium.com/analytics-vidhya/concept-of-alexnet-convolutional-neural-network-6e73b4f9ee30

Csatlakozó egyéb elkészült dokumentációk / fájlok / stb. jegyzéke:

A lefutatott kód itt található (a verzió változott azóta mióta a képek készültek):
https://colab.research.google.com/drive/1GxdcRbYB3dSGbshc5M51I3LXChk6_O?usp=sharing